Q1. Write code to develop a class called Circle which contains the following public attributes and methods. Decide appropriate attributes/parameters and return types.

- **Attributes: radius and color**
- **Methods:**
  - **setRadius(r) : sets radius value to r**
  - **getRadius() : returns radius value**
  - **setColor(c) : sets color name to c**
  - **getColor() : returns color name**
  - **getCircumference() : returns circumference of the circle**
  - **getArea() : returns area of the circle**

Instantiate necessary objects to test the functionality of this class.

```
class Circle():
    radius = 2
    color = "Blue"
    def setRadius(self, r):
        self.radius = r
    def getRadius(self):
        return self.radius
    def setColor(self, c):
        self.color = c
    def getColor(self):
        return self.color
    def getCircumference(self):
        circum = 2 * (3.14) * self.radius
        return circum
    def getArea(self):
        area = 3.14 * (self.radius ** 2)
        return area
c = Circle()
print("Radius:", c.getRadius())
print("Color:", c.getColor())
print("Circumference:", c.getCircumference())
print("Area:", c.getArea())

c.setRadius(5)
c.setColor("Red")
print("Radius:", c.getRadius())
print("Color:", c.getColor())
print("Circumference:", c.getCircumference())
print("Area:", c.getArea())
```

Q1.py - C:/Users/Shaya/Downloads/Q1.py (3.13.14)

File Edit Format Run Options Window Help

```
class Circle():
    radius = 2
    color = "Blue"
    def setRadius(self, r):
        self.radius = r
    def getRadius(self):
        return self.radius
    def setColor(self, c):
        self.color = c
    def getColor(self):
        return self.color
    def getCircumference(self):
        circum = 2 * (3.14) * self.radius
        return circum
    def getArea(self):
        area = 3.14 * (self.radius ** 2)
        return area
c = Circle()
print("\nRadius:", c.getRadius())
print("\nColor:", c.getColor())
print("\nCircumference:", c.getCircumference())
print("\nArea:", c.getArea())

c.setRadius(5)
c.setColor("Red")
print("\nRadius:", c.getRadius())
print("\nColor:", c.getColor())
print("\nCircumference:", c.getCircumference())
print("\nArea:", c.getArea())
```

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/Shaya/Downloads/Q1.py ==========

Radius: 2

Color: Blue

Circumference: 12.56

Area: 12.56

Radius: 5

Color: Red

Circumference: 31.400000000000002

Area: 78.5
>>>
```

Q2. Write code to develop a class BankAccount that supports the following methods:

- **withdraw() : takes an amount as input from user and withdraws it from the balance**
- **deposit() : takes an amount as input from user and adds it to the balance**
- **balance() : returns the balance on the account**

Make all the methods public and decide appropriate parameter(s) and return types wherever needed. Also create appropriate public attributes. Instantiate necessary objects to test the functionality of this class.

```
class BankAccount():

    account_balance = 0.0

    def withdraw(self):
        a = float(input("\nEnter Amount to Withdraw: $"))
        if a > self.account_balance:
            return "\nSorry! Insufficient funds"
        elif a <= 0:
            return "Withdrawal amount must be greater than Zero"
        else:
            self.account_balance -= a
            return "\nCongratulations! You have withdrawn: $" + str(a) + \
                   "\n\nThe new balance after withdrawal is: $" +
str(self.account_balance)

    def deposit(self):
        a = float(input("\nEnter Amount to be deposited: $"))

        if a <= 0:
            return "The amount to be deposited can not be Zero"
        else:
            self.account_balance += a
            return "\nCongratulations! You have deposited: $" + str(a) + \
                   "\n\nThe new Balance after deposit is: $" +
str(self.account_balance)


    def balance(self):
        return self.account_balance


b = BankAccount()

print("Initial Balance: $" + str(b.balance()))
print(str(b.deposit()))
print(b.withdraw())
print("\nYour current balance is: $" + str(b.balance()))
```

Q2. Write code to develop a class BankAccount that supports the following methods.py - C:\Users\Shaya\Downloads\Q2. Write code to develop a class BankAccount that supports the following methods.py (3.13.14)

File Edit Format Run Options Window Help

```
class BankAccount():

    account_balance = 0.0

    def withdraw(self):
        a = float(input("\nEnter Amount to Withdraw: $"))
        if a > self.account_balance:
            return "\nSorry! Insufficient funds"
        elif a <= 0:
            return "Withdrawal amount must be greater than Zero"
        else:
            self.account_balance -= a
            return "\nCongratulations! You have withdrawn: $" + str(a) + \
                   "\n\nThe new balance after withdrawal is: $" + str(self.account_balance)

    def deposit(self):
        a = float(input("\nEnter Amount to be deposited: $"))

        if a <= 0:
            return "The amount to be deposited can not be Zero"
        else:
            self.account_balance += a
            return "\nCongratulations! You have deposited: $" + str(a) + \
                   "\n\nThe new Balance after deposit is: $" + str(self.account_balance)


    def balance(self):
        return self.account_balance


b = BankAccount()

print("Initial Balance: $" + str(b.balance()))
print(str(b.deposit()))
print(b.withdraw())
print("\nYour current balance is: $" + str(b.balance()))
```

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\Shaya\Downloads\Q2. Write code to develop a class BankAccount that supports the following methods.py
Initial Balance: $0.0

Enter Amount to be deposited: $1000

Congratulations! You have deposited: $1000.0

The new Balance after deposit is: $1000.0

Enter Amount to Withdraw: $500

Congratulations! You have withdrawn: $500.0

The new balance after withdrawal is: $500.0

Your current balance is: $500.0
>>>
```

Q3. Repeat problem 2, making all attributes private. Make appropriate changes in the code wherever needed

```
class BankAccount():

    __account_balance = 0.0

    def withdraw(self):
        a = float(input("\nEnter Amount to Withdraw: $"))
        if a > self.__account_balance:
            return "\nSorry! Insufficient funds"
        elif a <= 0:

            return "Withdrawal amount must be greater than Zero"
        else:

            self.__account_balance -= a

            return "\nCongratulations! You have withdrawn: $" + str(a) + \
                   "\n\nThe new balance after withdrawal is: $" +
str(self.__account_balance)

    def deposit(self):
        a = float(input("\nEnter Amount to be deposited: $"))
        if a <= 0:
            return "The amount to be deposited can not be Zero"
        else:
            self.__account_balance += a
            return "\nCongratulations! You have deposited: $" + str(a) + \
                   "\n\nThe new Balance after deposit is: $" +
str(self.__account_balance)
    def balance(self):
        return self.__account_balance


b = BankAccount()
print("Initial Balance: $" + str(b.balance()))
print(str(b.deposit()))
print(b.withdraw())
print("\nYour current balance is: $" + str(b.balance()))
```

Q3. Repeat problem making all attributes private.py - C:\Users\Shaya\Downloads\Q3. Repeat problem making all attributes private.py (3.13.14)

File Edit Format Run Options Window Help

```
class BankAccount():

    __account_balance = 0.0

    def withdraw(self):
        a = float(input("\nEnter Amount to Withdraw: $"))
        if a > self.__account_balance:
            return "\nSorry! Insufficient funds"
        elif a <= 0:

            return "Withdrawal amount must be greater than Zero"
        else:

            self.__account_balance -= a

            return "\nCongratulations! You have withdrawn: $" + str(a) + \
                   "\n\nThe new balance after withdrawal is: $" + str(self.__account_balance)

    def deposit(self):
        a = float(input("\nEnter Amount to be deposited: $"))
        if a <= 0:
            return "The amount to be deposited can not be Zero"
        else:
            self.__account_balance += a
            return "\nCongratulations! You have deposited: $" + str(a) + \
                   "\n\nThe new Balance after deposit is: $" + str(self.__account_balance)
    def balance(self):
        return self.__account_balance


b = BankAccount()
print("Initial Balance: $" + str(b.balance()))
print(str(b.deposit()))
print(b.withdraw())
print("\nYour current balance is: $" + str(b.balance()))
```

Ln: 34 Col: 19

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\Shaya\Downloads\Q3. Repeat problem making all attributes private.py
Initial Balance: $0.0

Enter Amount to be deposited: $55

Congratulations! You have deposited: $55.0

The new Balance after deposit is: $55.0

Enter Amount to Withdraw: $20

Congratulations! You have withdrawn: $20.0

The new balance after withdrawal is: $35.0

Your current balance is: $35.0
>>>
```

Ln: 20 Col: 0

Q4. Write code to implement class Worker that supports two private attributes hoursWorked and wageRate, and the following public methods:

- **setHoursWorked(h): sets hoursWorked to h**
- **changeRate(r): changes the worker's pay rate to the new hourly rate r**
- **pay(): returns the pay as product of hoursWorked and wageRate**

Instantiate necessary objects to test the functionality of this class.

```
class Worker:

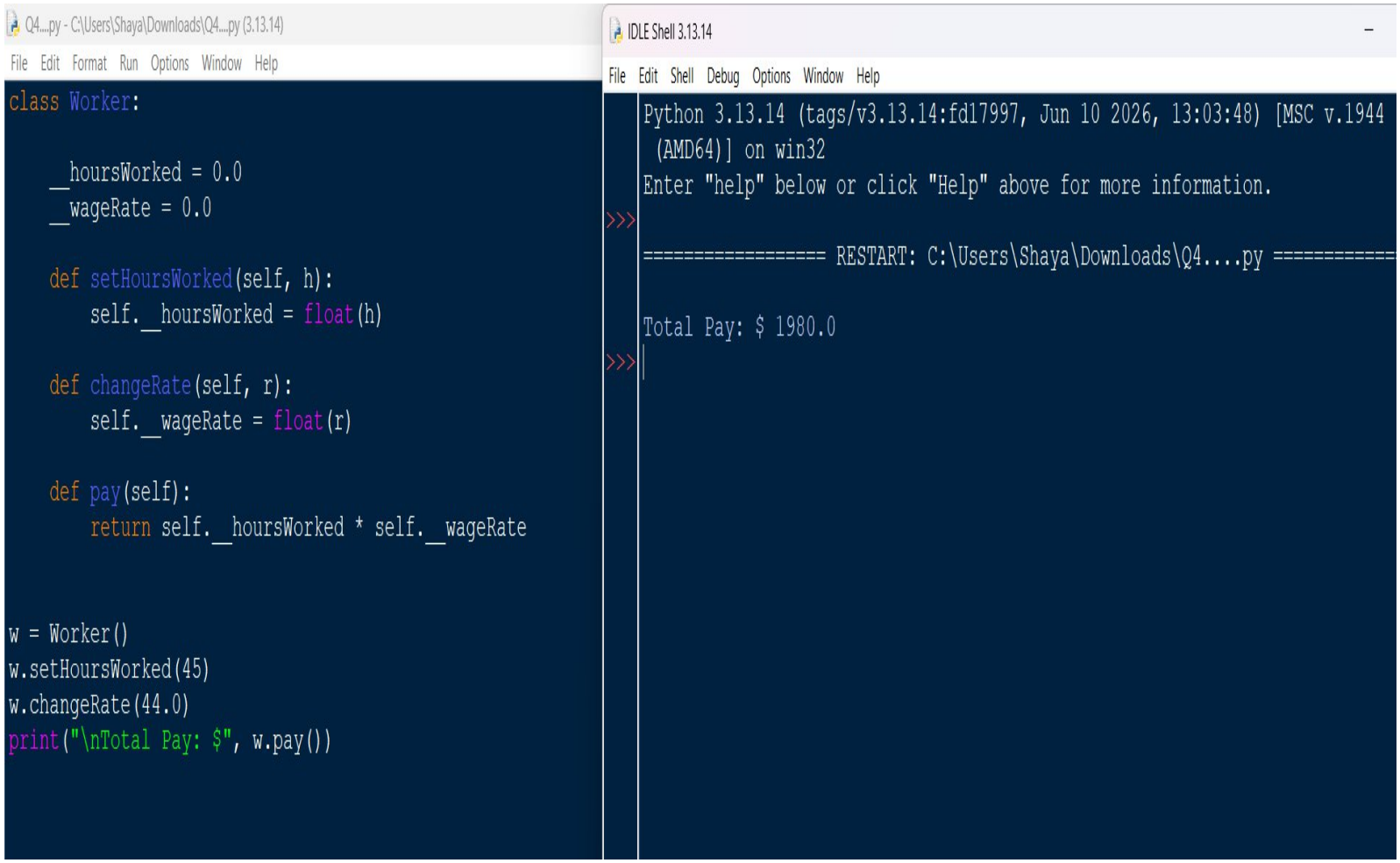
    __hoursWorked = 0.0
    __wageRate = 0.0

    def setHoursWorked(self, h):
        self.__hoursWorked = float(h)

    def changeRate(self, r):
        self.__wageRate = float(r)

    def pay(self):
        return self.__hoursWorked * self.__wageRate


w = Worker()
w.setHoursWorked(45)
w.changeRate(44.0)
print("\nTotal Pay: $", w.pay())
```



Q5. Repeat problem 4, adding constructor to initialize the attributes. Also set attributes' default values.

```
class Worker():

    def __init__(self, h=0.0, r=0.0):
        self.setHoursWorked(h)
        self.changeRate(r)

    def setHoursWorked(self, h):
        self.__hoursWorked = float(h)

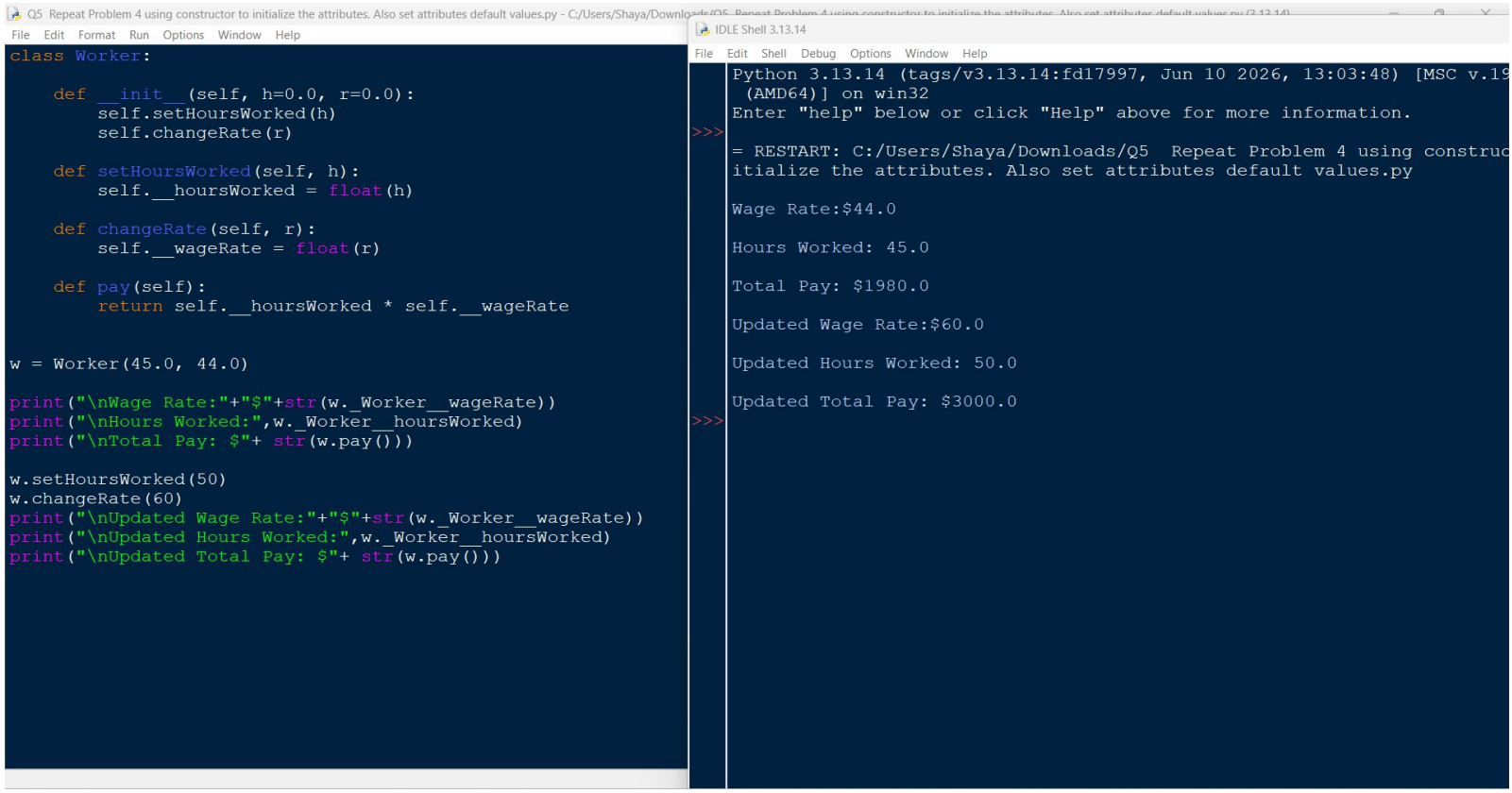
    def changeRate(self, r):
        self.__wageRate = float(r)

    def pay(self):
        return self.__hoursWorked * self.__wageRate


w = Worker(45.0, 44.0)

print("\nWage Rate:"+"$"+str(w._Worker__wageRate))
print("\nHours Worked:",w._Worker__hoursWorked)
print("\nTotal Pay: $"+ str(w.pay()))

w.setHoursWorked(50)
w.changeRate(60)
print("\nUpdated Wage Rate:"+"$"+str(w._Worker__wageRate))
print("\nUpdated Hours Worked:",w._Worker__hoursWorked)
print("\nUpdated Total Pay: $"+ str(w.pay()))
```



Q6. Write a code to develop a class named Vehicles having three private attributes noOfWheels, color and modelNo; define related constructor and public getter/setter methods. Use appropriate types for all attributes, method parameters and return values wherever needed. Instantiate necessary objects to test the functionality of the class.

```
class Vehicles():

    def __init__(self,noOfWheels,color,modelNo):
        self.__noOfWheels=noOfWheels
        self.__color=color
        self.__modelNo=modelNo
    def setNoOfWheels(self,noOfWheels):
        self.__noOfWheels=noOfWheels
    def getNoOfWheels(self):
        return self.__noOfWheels
    def setColor(self,color):
        self.__color=color
    def getColor(self):
        return self.__color
    def setModelNo(self,modelNo):
        self.__modelNo=modelNo
    def getModelNo(self):
        return self.__modelNo

v=Vehicles(4,"Red",1987)
print("\nDefault No. of Wheels",v.getNoOfWheels())
print("Default Color:",v.getColor())
print("Default Model No:",v.getModelNo())

v.setNoOfWheels(5)
v.setColor("Black")
v.setModelNo(2015)
print("\nSet No.of Wheels:",v.getNoOfWheels())
print("Set Color:",v.getColor())
print("Set Model No:",v.getModelNo())
```

Q6.py - C:\Users\Shaya\Downloads\Q6.py (3.13.14)

File Edit Format Run Options Window Help

```
class Vehicles():

    def __init__(self,noOfWheels,color,modelNo):
        self.__noOfWheels=noOfWheels
        self.__color=color
        self.__modelNo=modelNo
    def setNoOfWheels(self,noOfWheels):
        self.__noOfWheels=noOfWheels
    def getNoOfWheels(self):
        return self.__noOfWheels
    def setColor(self,color):
        self.__color=color
    def getColor(self):
        return self.__color
    def setModelNo(self,modelNo):
        self.__modelNo=modelNo
    def getModelNo(self):
        return self.__modelNo

v=Vehicles(4,"Red",1987)
print("\nDefault No. of Wheels",v.getNoOfWheels())
print("Default Color:",v.getColor())
print("Default Model No:",v.getModelNo())

v.setNoOfWheels(5)
v.setColor("Black")
v.setModelNo(2015)
print("\nSet No.of Wheels:",v.getNoOfWheels())
print("Set Color:",v.getColor())
print("Set Model No:",v.getModelNo())
```

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64
 (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>

========== RESTART: C:\Users\Shaya\Downloads\Q6.py ==========

Default No. of Wheels 4
Default Color: Red
Default Model No: 1987

Set No.of Wheels: 5
Set Color: Black
Set Model No: 2015
>>>
```

Q7. Write code to develop a class Engine having private attributes engineNo and dateOfManufacture, along with appropriate constructor and public getter/setter methods. Decide appropriate types for all attributes, method parameters and return values for this new class.

```
class Engine():

    def __init__(self,engineNo,dateOfManufacture):
        self.__engineNo=engineNo
        self.__dateOfManufacture=dateOfManufacture
    def setEngineNo(self,engineNo):
        self.__engineNo=engineNo
    def getEngineNo(self):
        return self.__engineNo
    def setDateOfManufacture(self,dateOfManufacture):
        self.__dateOfManufacture=dateOfManufacture
    def getDateOfManufacture(self):
        return self.__dateOfManufacture

e=Engine(1112151515, "19-9-2026")
print("\nDefault Engine No:",e.getEngineNo())
print("Default Date of Manufacture:",e.getDateOfManufacture())

e.setEngineNo(2564831651515151)
e.setDateOfManufacture("29-12-2003")
print("\nUpdated Engine No:",e.getEngineNo())
print("Updated Date of Manufacture:",e.getDateOfManufacture())
```

